SESIÓN 2 · PREPARACIÓN DEL ENTORNO

Guía de instalación y configuración del entorno

VS Code, navegador, Git y GitHub, Node.js y TypeScript para programar en el entorno cliente

Módulo	CSTI12004 Programación web en el entorno cliente
Sistema operativo	Windows 10 u 11 de 64 bits (con notas para macOS y Linux)
Duración estimada	100 minutos en la sesión 2 (miércoles 11/11, virtual, en su computadora) · se repite en la sesión 3 en el laboratorio · Parte 2 antes de la sesión 21
Persona facilitadora	Giovanni Antonio Coto Calderón
Persona estudiante	_____

Antes de empezar

- Descargue todo **solo de los sitios oficiales** que indica esta guía. Evite páginas de descargas de terceros.
- Necesita conexión a internet y unos **2 GB** libres en el disco.
- En una computadora del laboratorio del INA es posible que no tenga permisos de administrador. Si un instalador lo pide, **avise a la persona facilitadora** en lugar de buscar otra versión.
- Al final hay una **lista de verificación**: márquela conforme avance.

1. Qué vamos a instalar y cuándo

Herramienta	Para qué la usamos	Cuándo	Sitio oficial
Google Chrome o Microsoft Edge	Ejecutar las páginas y depurar con las herramientas del desarrollador (F12).	Parte 1 · S02	google.com/chrome · Edge ya viene en Windows
Visual Studio Code	Editor de código de todo el módulo.	Parte 1 · S02	code.visualstudio.com
Extensiones de VS Code	Servidor local, formato automático e idioma español.	Parte 1 · S02	Dentro de VS Code
Git + cuenta de GitHub	Control de versiones: cada práctica y proyecto se entrega con su repositorio.	Parte 1 · S02	git-scm.com · github.com
Node.js LTS (incluye npm)	Instalar TypeScript y Angular.	Parte 2 · antes de S26	nodejs.org
TypeScript	Compilador tsc de la UA4.	Parte 2 · S26	Se instala con npm
Angular CLI	Crear y ejecutar proyectos Angular.	Parte 2 · S32	Se instala con npm

2. Navegador

1. Use **Google Chrome** o **Microsoft Edge** actualizados. Los dos usan el mismo motor (Chromium) y tienen las mismas herramientas.
2. Para comprobar que está actualizado: menú ⋮ → **Ayuda** → **Información de Google Chrome** (en Edge: **…** → **Ayuda y comentarios** → **Acerca de Microsoft Edge**).
3. Abra cualquier página y presione **F12**. Debe abrirse un panel con pestañas como **Elements** y **Console**: son las herramientas del desarrollador (DevTools) que usará todos los días.

Opcional: instale también **Firefox** para probar sus páginas en un navegador con otro motor (útil en la UA3, accesibilidad).

3. Visual Studio Code

3.1 Descarga e instalación

1. Entre a **code.visualstudio.com** y presione **Download for Windows**. Se descarga el instalador de usuario (**User Installer, x64**), que no necesita permisos de administrador.
2. Ejecute el archivo descargado (VSCodeUserSetup-x64-…exe) y acepte el acuerdo de licencia.
3. En la pantalla **Seleccione las tareas adicionales**, marque las cinco casillas:

Casilla del instalador	Por qué marcarla
Crear un icono en el escritorio	Acceso rápido.
Agregar la acción «Abrir con Code» al menú contextual de archivo	Clic derecho en un archivo → Abrir con Code.
Agregar la acción «Abrir con Code» al menú contextual de directorio	Clic derecho en una carpeta → Abrir con Code. **Es la forma recomendada de abrir cada práctica.**
Registrar Code como editor para tipos de archivo admitidos	Los .html, .css y .js se abren con VS Code.
Agregar a PATH	Permite usar el comando **code** desde la terminal. **Indispensable.**

4. Presione **Instalar** y, al terminar, **Finalizar** con la casilla «Ejecutar Visual Studio Code» marcada.

macOS y Linux

- **macOS:** descargue el .zip, arrastre Visual Studio Code a Aplicaciones. Para el comando code: Ctrl+Shift+P (Cmd+Shift+P) → «Shell Command: Install 'code' command in PATH».
- **Linux (Ubuntu/Debian):** descargue el paquete .deb y ejecute sudo apt install ./code_*.deb.

3.2 Las partes de VS Code

Parte	Ubicación	Para qué sirve
Barra de actividades	Columna de iconos a la izquierda	Explorador (Ctrl+Shift+E), Buscar, Control de código fuente (Git), Extensiones (Ctrl+Shift+X).
Explorador	Panel lateral	Muestra los archivos de la carpeta abierta.

Parte	Ubicación	Para qué sirve
Editor	Centro	Donde se escribe el código; cada archivo abre en una pestaña. Un punto ● en la pestaña indica cambios sin guardar.
Terminal	Abajo (menú Terminal → Nueva terminal; atajo Ctrl+ñ o Ctrl+` según el teclado)	Ejecuta comandos de Git, Node.js y TypeScript sin salir del editor.
Barra de estado	Franja inferior	Rama de Git, errores, codificación (UTF-8) y el botón **Go Live** de Live Server.

Regla de oro: abra carpetas, no archivos

Trabaje siempre con **Archivo → Abrir carpeta…** (o clic derecho en la carpeta → Abrir con Code). Si abre un archivo suelto, Live Server, Git y el autocompletado entre archivos no funcionan bien.

4. Extensiones

Abra la vista de **Extensiones** (Ctrl+Shift+X), busque cada una por su nombre, verifique el **publicador** (evite copias con el mismo nombre) y presione **Install**.

Extensión	Publicador · identificador	Para qué	Cuándo
Spanish Language Pack for Visual Studio Code	Microsoft · MS-CEINTL.vscode-language-pack-es	Menús en español. Al instalarla, VS Code pide reiniciar.	Hoy (opcional)
Live Server	Ritwick Dey · ritwickdey.LiveServer	Servidor local: abre la página en el navegador y la recarga sola al guardar. Necesario para fetch, JSON y Web Workers.	Hoy
Prettier - Code formatter	Prettier · esbenp.prettier-vscode	Ordena la sangría y el formato del código al guardar.	Hoy
Error Lens	Alexander · usernamehw.errorlens	Muestra los errores al lado de la línea donde ocurren.	Hoy (recomendada)
axe Accessibility Linter	Deque Systems · deque-systems.vscode-axe-linter	Señala problemas de accesibilidad en el HTML.	UA3 · S22
Angular Language Service	Angular · Angular.ng-template	Autocompletado y errores en plantillas de Angular.	UA4 · S32

Opción rápida: instalar desde la terminal

Si el comando **code** quedó en el PATH, puede instalar las extensiones de la Parte 1 en un solo paso. Abra una terminal (en VS Code: Terminal → Nueva terminal) y escriba:

Terminal

```
code --install-extension MS-CEINTL.vscode-language-pack-es
code --install-extension ritwickdey.LiveServer
code --install-extension esbenp.prettier-vscode
code --install-extension usernamehw.errorlens
```

Alternativa a Live Server: la extensión **Live Preview** de Microsoft (ms-vscode.live-server) muestra la página dentro de VS Code. En el curso usamos Live Server porque abre el navegador real, donde están las DevTools.

5. Configuración de VS Code

Esta configuración hace que todos trabajemos igual: mismo tamaño de sangría, formato automático al guardar y codificación UTF-8 (para que las tildes y la ñ se vean bien).

1. Presione **Ctrl+Shift+P** para abrir la paleta de comandos.
2. Escriba **settings json** y elija **Preferencias: Abrir configuración de usuario (JSON)** (en inglés: Preferences: Open User Settings (JSON)).
3. Si el archivo trae contenido, conserve lo que tenga y agregue las líneas nuevas antes de la llave de cierre }, separadas por comas. Si está vacío, pegue todo el bloque.
4. Guarde con **Ctrl+S**. Si algo quedó mal escrito, VS Code subraya en rojo la línea con el error.

settings.json (configuración de usuario)

```
{
  "editor.fontSize": 16,
  "editor.tabSize": 2,
  "editor.wordWrap": "on",
  "editor.linkedEditing": true,
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "files.autoSave": "onFocusChange",
  "files.encoding": "utf8",
  "files.eol": "\n",
  "terminal.integrated.defaultProfile.windows": "Git Bash",
  "liveServer.settings.donotShowInfoMsg": true,
  "git.autofetch": true
}
```

Línea	Qué hace
`editor.fontSize: 16`	Letra más legible (ajústela a su gusto).
`editor.tabSize: 2`	Sangría de 2 espacios, la convención de HTML, CSS y JavaScript (en C usaba 4).
`editor.wordWrap`	Las líneas largas se ajustan a la ventana: no hay que desplazarse a la derecha.
`editor.linkedEditing`	Al cambiar se cambia también .
`editor.formatOnSave + defaultFormatter`	Al guardar, Prettier ordena el código.
`files.autoSave: onFocusChange`	Guarda solo al cambiar de ventana (por ejemplo, al pasar al navegador). Igual conviene el hábito de Ctrl+S.
`files.encoding / files.eol`	UTF-8 y saltos de línea uniformes: evita problemas de tildes y cambios fantasma en Git.
`terminal.integrated.defaultProfile.windows`	La terminal de VS Code abre Git Bash (se instala en la sección 6): los mismos comandos funcionan en Windows, macOS y Linux.
`git.autofetch`	VS Code revisa periódicamente si hay cambios en GitHub.

6. Git y GitHub

6.1 Instalar Git

1. Entre a **git-scm.com**, sección **Downloads** → **Windows**, y descargue **64-bit Git for Windows Setup**.
2. Ejecute el instalador. Puede dejar casi todo por defecto, **excepto estas tres pantallas**:

Pantalla del instalador	Elija
Choosing the default editor used by Git	**Use Visual Studio Code as Git's default editor**
Adjusting the name of the initial branch in new repositories	**Override the default branch name** y escriba **main**
Adjusting your PATH environment	**Git from the command line and also from 3rd-party software** (recomendada)

3. Al terminar, **cierre y vuelva a abrir VS Code** para que reconozca Git.

macOS: abra la Terminal y escriba git --version; si no está, el sistema ofrece instalarlo. **Linux:** sudo apt install git.

6.2 Configurar su identidad

Abra la terminal de VS Code (**Terminal** → **Nueva terminal**). Debe decir **bash** en la esquina de la terminal. Escriba cada línea con **sus datos** (use el mismo correo de su cuenta de GitHub):

Terminal · Git Bash

```
git config --global user.name "Nombre Apellido"
git config --global user.email "su_correo@ejemplo.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --global --list
```

La última línea muestra la configuración: revise que el nombre y el correo estén bien escritos.

6.3 Cuenta de GitHub

Si ya tiene la cuenta del módulo anterior, **use la misma**. Si no, créela en **github.com** → **Sign up** con un nombre de usuario profesional: será parte de su portafolio.

6.4 Repositorio del módulo (portafolio de evidencias)

Igual que en el módulo anterior, cada módulo tiene **su propio repositorio**, organizado por unidad y sesión, con un **README** principal que funciona como portada, **registro de evidencias** y **bitácora**. Este repositorio es el que se entrega en el archivo **repositorio_url.txt** de cada actividad de comprobación (el indicador 22 de la AC N.º 1 revisa el historial de versiones).

Paso 1 · Crear el repositorio en GitHub

1. En github.com: botón **+** (arriba a la derecha) → **New repository**.
2. **Repository name:** CSTI12004-Nombre-Apellido (sin tildes ni ñ; por ejemplo, CSTI12004-Maria-Nunez).
3. **Description:** Programación web en el entorno cliente · INA · 2026.
4. Visibilidad: **Public** (la persona facilitadora debe poder verlo).

5. Marque **Add a README file**. En **.gitignore** y **license** deje **None**: los crearemos nosotros.
6. Presione **Create repository**. Luego, botón verde **Code** → pestaña **HTTPS** → copie el URL (termina en .git).

Paso 2 · Clonarlo en la carpeta INA

Use la misma carpeta del módulo anterior, **C:\INA**. Su repositorio anterior puede quedarse ahí: cada repositorio es una carpeta independiente. Abra **Git Bash** y escriba:

Terminal · Git Bash

```
mkdir -p /c/INA            # crea C:\INA si no existe
cd /c/INA
git clone https://github.com/SU_USUARIO/CSTI12004-Nombre-Apellido.git
cd CSTI12004-Nombre-Apellido
code .                     # abre el repositorio en VS Code
```

Desde ahora, **siempre abra esta carpeta** en VS Code (Archivo → Abrir carpeta… → C:\INA\CSTI12004-Nombre-Apellido). La barra de estado debe mostrar la rama **main**.

Paso 3 · Crear la estructura inicial

En la terminal de VS Code (ya dentro del repositorio) cree las carpetas de la primera sesión:

Terminal · Git Bash

```
mkdir -p UA1/S01 UA1/S02/capturas
touch .gitignore UA1/S01/README.md UA1/S02/README.md
```

Copie en **UA1/S01** su cuadro comparativo (la tarea de la sesión 1), con el nombre **S01_Cuadro_comparativo_Nombre_Apellido.docx**.

La estructura completa, que irá creciendo sesión a sesión, es:

Estructura del repositorio

```
CSTI12004-Nombre-Apellido/
├── README.md                  ← portada, registro de evidencias y bitácora
├── .gitignore                 ← archivos que Git no debe subir
├── UA1/
│   ├── S01/
│   │   ├── README.md
│   │   └── S01_Cuadro_comparativo_Nombre_Apellido.docx
│   ├── S02/
│   │   ├── README.md          ← evidencia de la sesión
│   │   ├── capturas/
│   │   │   └── prueba_entorno.png
│   │   └── prueba_entorno/
│   │       ├── index.html
│   │       └── app.js
│   ├── S03/ … S09/
├── UA2/
│   ├── S09/ … S21/
│   └── Proyecto01/            ← actividad de comprobación N.° 1
├── UA3/
│   ├── S19/ … S28/
│   └── Proyecto02/            ← actividad de comprobación N.° 2
└── UA4/
    ├── S26/ … S38/
    └── Proyecto03/            ← actividad de comprobación N.° 3
```

Elemento	Regla
`Carpetas UA1 … UA4`	Una por unidad de aprendizaje. Se crean cuando empieza la unidad.
`Carpetas S01 … S38`	Una por sesión, con dos dígitos (S01, no S1). Las sesiones compartidas (S09, S19, S26 y S28) tienen carpeta en las dos unidades.
`Proyecto01 … Proyecto03`	Proyectos de las actividades de comprobación, dentro de la unidad que los evalúa.
`capturas/`	Imágenes de evidencia de la sesión (.png). Win+Shift+S toma una captura de una zona de la pantalla.
`Nombres de archivos`	Minúsculas, sin tildes, sin ñ y sin espacios: calculadora_iva.js, no Calculadora IVA.js.

Paso 4 · El archivo .gitignore

Le indica a Git qué no debe subir. Hoy casi no se nota; en la UA4 evita subir la carpeta **node_modules** de Angular, que pesa cientos de megabytes. Abra **.gitignore** y escriba:

.gitignore

```
# Dependencias y compilados (UA4: TypeScript y Angular)
node_modules/
dist/
.angular/

# Registros y variables de entorno
*.log
.env

# Archivos del sistema operativo
Thumbs.db
desktop.ini
.DS_Store

# Archivos temporales de Office
~$*
```

Paso 5 · El README principal

Abra **README.md** (el que creó GitHub), **borre su contenido** y escriba esta plantilla con **sus datos**. Puede copiarla del archivo README.md que comparte la persona facilitadora. Para ver cómo se mostrará en GitHub, presione **Ctrl+Shift+V** (vista previa de Markdown).

README.md (raíz del repositorio) · versión resumida

```
# CSTI12004 · Programación web en el entorno cliente

**Persona estudiante:** Nombre Apellido1 Apellido2
**Programa:** CSTI14001 Desarrollo de páginas web · Instituto Nacional de Aprendizaje (INA)
**Persona facilitadora:** Giovanni Antonio Coto Calderón
**Fechas del módulo:** del 10/11/2026 al 16/02/2027

Portafolio de evidencias del módulo. Contiene las prácticas de cada sesión, los proyectos de las
actividades de comprobación y la bitácora de aprendizaje.

## Estructura

```
CSTI12004-Nombre-Apellido/
```
```

```
├── README.md          ← este archivo: registro de evidencias y bitácora
├── .gitignore
├── UA1/   S01 … S09   Elaboración de componentes de software (JavaScript)
├── UA2/   S09 … S21   Manipulación del DOM e interactividad web · Proyecto01
├── UA3/   S19 … S28   Accesibilidad web · Proyecto02
└── UA4/   S26 … S38   TypeScript y Angular · Proyecto03
```

Cada carpeta de sesión tiene su propio `README.md` con los archivos del día, capturas, conclusión y autoevaluación.

Registro de evidencias

☐ Pendiente · ▨ En proceso · ☑ Completa

Sesión	Fecha	Tema	Carpeta	Estado
S01	10/11/2026	Bienvenida, encuadre y diagnóstico	[UA1/S01](UA1/S01)	☐
S02	11/11/2026	Entorno de trabajo, MVC y JavaScript	[UA1/S02](UA1/S02)	☐
S03	12/11/2026	Entorno en el laboratorio y mi primer script	[UA1/S03](UA1/S03)	☐
…		(una fila por sesión, hasta S30)		☐
S31	17/02/2027	TypeScript: namespaces, módulos y librerías externas	[UA4/S31](UA4/S31)	☐
S32	18/02/2027	Angular: entorno, componentes y enrutamiento	[UA4/S32](UA4/S32)	☐
S33	19/02/2027	Angular: data binding y pipes	[UA4/S33](UA4/S33)	☐
S34	22/02/2027	Angular: directivas, @Input, @Output y @ViewChild	[UA4/S34](UA4/S34)	☐
S35	23/02/2027	Angular: servicios, asincronía, RxJS y HTTP	[UA4/S35](UA4/S35)	☐
S36	24/02/2027	Angular: formularios · Actividad de comprobación N.° 3 (inicio)	UA4/Proyecto03	☐
S37	25/02/2027	Actividad de comprobación N.° 3 · 2.ª oportunidad (inicio)	[UA4/S37](UA4/S37)	☐
S38	26/02/2027	2.ª oportunidad AC N.° 3 · Cierre del módulo	[UA4/S38](UA4/S38)	☐

Actividades de comprobación

AC	Proyecto	Carpeta	Estado
1	Manipulación del DOM e interactividad web (UA1 + UA2)	UA2/Proyecto01	☐
2	Accesibilidad web	UA3/Proyecto02	☐
3	Desarrollo de página web empleando framework frontend	UA4/Proyecto03	☐

Bitácora

Fecha	Sesión	Qué aprendí o hice	Dificultad y cómo la resolví
10/11/2026	S01	Bienvenida, encuadre del módulo y diagnóstico. Tarea: cuadro comparativo de C, JavaScript y TypeScript.	
11/11/2026	S02	Instalé el entorno (VS Code, Live Server, Git), creé este repositorio y estudié MVC y qué es JavaScript.	

Convenciones

```
- Nombres de archivos y carpetas **sin tildes, sin ñ y sin espacios** (`calculadora_iva.js`).
- `git pull` al iniciar la sesión y `git push` al terminarla.
- Un commit por avance, con la sesión al inicio: `S02: agrega operadores.js`.
- Las capturas de pantalla van en la carpeta `capturas/` de cada sesión.
```

El archivo **README.md** completo trae ya escritas las 38 filas del registro (S01 a S38) con su tema y carpeta: solo hay que cambiar el nombre.

Cómo se mantiene el README principal

- **Registro de evidencias:** al terminar cada sesión, escriba la fecha y cambie ☐ por ☑ (o ▨ si quedó a medias).
- **Bitácora:** agregue una fila por sesión: qué aprendió o hizo y qué dificultad tuvo. Una o dos líneas bastan.
- Los enlaces de la columna **Carpeta** funcionan en GitHub cuando la carpeta ya existe.

Paso 6 · El README de cada sesión

Cada carpeta de sesión lleva su propio **README.md**. La S01 tuvo un trabajo sencillo, así que su README es corto:

UA1/S01/README.md

```
# S01 · Bienvenida, encuadre y diagnóstico

**Fecha:** 10/11/2026 (presencial) · **Unidad:** UA1 Elaboración de componentes de software

## Qué hice

- Encuadre del módulo, fechas importantes y acuerdos de convivencia.
- Diagnóstico general del módulo.
- Tarea: cuadro comparativo de C, JavaScript y TypeScript.

## Archivos

| Archivo | Descripción |
|---|---|
| S01_Cuadro_comparativo_Nombre_Apellido.docx | Cuadro comparativo (tarea) |

## Conclusión

Escriba aquí, en dos o tres líneas, la diferencia más importante que encontró entre C y JavaScript.
```

El de la S02 es el modelo completo que seguirán todas las sesiones:

UA1/S02/README.md

```
# S02 · Entorno de trabajo, MVC y JavaScript

**Fecha:** 11/11/2026 (virtual) · **Unidad:** UA1 Elaboración de componentes de software

## Qué hice

- Puesta en común del cuadro comparativo de C, JavaScript y TypeScript.
- Instalación y configuración del entorno; creación de este repositorio.
```

```
 9  - Prueba del entorno con Live Server.
10  - Conceptos: MVC, lenguajes compilados e interpretados, sintaxis y semántica.
11
12  ## Archivos
13
14  | Archivo | Descripción |
15  |---|---|
16  | [prueba_entorno/index.html](prueba_entorno/index.html) | Página de prueba del
entorno |
17  | [prueba_entorno/app.js](prueba_entorno/app.js) | Script enlazado a la página
|
18
19  ## Evidencia
20
21  ![Prueba del entorno en el navegador](capturas/prueba_entorno.png)
22
23  ## Conclusión
24
25  Escriba aquí, en dos o tres líneas, la diferencia más importante que encontró
entre C y JavaScript.
26
27  ## Autoevaluación
28
29  | Indicador | Sí | Aún no |
30  |---|---|---|
31  | Instalé y configuré VS Code con sus extensiones. | | |
32  | Creé mi repositorio y subí cambios con `git push`. | | |
33  | Abro mis páginas con Live Server y reviso la consola (F12). | | |
34  | Explico la diferencia entre un lenguaje compilado y uno interpretado. | | |
```

Para mostrar una imagen en Markdown se escribe **![texto alternativo](ruta/de/la/imagen.png)**. El texto alternativo describe la imagen: lo retomaremos en la UA3, accesibilidad.

Paso 7 · Primer commit y push

Terminal · Git Bash

```
1  git status          # muestra archivos nuevos y modificados
2  git add README.md .gitignore UA1/S01
3  git commit -m "S02: estructura inicial del portafolio"
4  git push
```

Iniciar sesión en GitHub la primera vez

La primera vez que haga **git push**, Git for Windows abre una ventana para **iniciar sesión con el navegador** (Git Credential Manager). Elija **Sign in with your browser** y autorice. GitHub ya no acepta la contraseña de la cuenta en la terminal.

Rutina de cada sesión

Momento	Comandos y acciones
Al iniciar	cd /c/INA/CSTI12004-Nombre-Apellido · git pull · mkdir -p UA1/S02/capturas (la carpeta del día)
Durante la sesión	Un commit por ejercicio terminado: git add . · git commit -m "S02: agrega operadores.js"
Al cerrar	Complete el README de la sesión (capturas, conclusión, autoevaluación); actualice el registro (☑) y la bitácora del README principal; git add . · git commit -m "S02: README y bitácora" · git push

La fecha y hora de cada commit son evidencia de que trabajó durante la sesión: **no deje todo para el final**.

7. Prueba final de la Parte 1

Esta prueba confirma que VS Code, Live Server, el navegador y Git funcionan juntos. Todavía no necesita entender el código JavaScript: lo estudiaremos en las próximas sesiones.

1. En el Explorador de VS Code cree las carpetas **UA1/S02/prueba_entorno** (icono de carpeta nueva).
2. Dentro de prueba_entorno cree el archivo **index.html** y escriba (o pegue) este código:

UA1/S02/prueba_entorno/index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Prueba del entorno</title>
</head>
<body>
  <h1>Mi entorno de trabajo</h1>
  <p id="mensaje">Si este texto no cambia, JavaScript no está enlazado.</p>
  <script src="app.js"></script>
</body>
</html>
```

3. En la misma carpeta cree **app.js**:

UA1/S02/prueba_entorno/app.js

```
// Prueba del entorno · CSTI12004
console.log("¡Hola desde app.js! El entorno funciona.");

const mensaje = document.getElementById("mensaje");
mensaje.textContent = "JavaScript está enlazado correctamente.";
```

4. Guarde los dos archivos (**Ctrl+S**). Prettier puede reacomodar espacios: es normal.
5. Clic derecho sobre index.html → **Open with Live Server** (o el botón **Go Live** de la barra de estado). Se abre el navegador en una dirección como **http://127.0.0.1:5500/UA1/S02/prueba_entorno/index.html**.
6. La página debe decir **«JavaScript está enlazado correctamente.»** Presione **F12** → pestaña **Console**: debe aparecer **¡Hola desde app.js! El entorno funciona.**
7. Cambie el texto del h1 en index.html y guarde: la página se recarga sola.
8. Tome una captura del navegador con la página y la consola abiertas (**Win+Shift+S**) y guárdela como **UA1/S02/capturas/prueba_entorno.png**.
9. Complete **UA1/S02/README.md** con la plantilla del paso 6 (sección 6.4). En el README principal marque la S01 y la S02 con ☑ y revise la fila de la bitácora.
10. Suba todo a GitHub desde la terminal:

Terminal · Git Bash (dentro de la carpeta del repositorio)

```
git status
git add UA1/S02/prueba_entorno
git commit -m "S02: agrega prueba del entorno"
git add .
git commit -m "S02: README de la sesión y registro de evidencias"
git push
```

11. Recargue el repositorio en GitHub: en la portada se ve el README con el registro y la bitácora; entre a **UA1** → **S02** y verá su README con la captura. **¡Listo!**

8. Repetir la Parte 1 en la computadora del laboratorio (sesión 3)

En la sesión 3 (jueves 12/11, presencial) se repite la instalación y la configuración en la computadora del laboratorio que usará en las sesiones presenciales. El repositorio **ya existe** en GitHub: **no se crea otro**, se **clona**.

Paso	En la computadora del laboratorio	Sección de esta guía
1	Revise el navegador y las DevTools (F12).	2
2	Abra VS Code. Si no está instalado, instálelo con el **User Installer** (no pide permisos de administrador).	3.1
3	Instale las extensiones Live Server, Prettier y Error Lens (y el paquete de español, si lo desea).	4
4	Pegue la configuración del curso en settings.json.	5
5	Compruebe Git con git --version. Si no está instalado, pida apoyo a la persona facilitadora: el instalador requiere permisos de administrador.	6.1
6	Configure su identidad (vea el recuadro de abajo).	6.2
7	Clone su repositorio en C:\INA y ábralo en VS Code.	Comandos de abajo
8	Abra UA1/S02/prueba_entorno/index.html con Live Server: debe funcionar igual que en su casa.	7

Terminal · Git Bash (computadora del laboratorio)

```
mkdir -p /c/INA
cd /c/INA
git clone https://github.com/SU_USUARIO/CSTI12004-Nombre-Apellido.git
cd CSTI12004-Nombre-Apellido
code .
```

Si el repositorio ya estaba clonado en esa computadora (por ejemplo, si la usó en otra sesión), no lo clone de nuevo: entre a la carpeta y ejecute **git pull** para traer lo último.

Identidad en una computadora compartida

Si la computadora del laboratorio la usan varias personas con el **mismo usuario de Windows**, configure su identidad **solo para su repositorio** (sin --global), dentro de la carpeta del repositorio:

git config user.name "Nombre Apellido" y **git config user.email "su_correo@ejemplo.com"**

Si cada estudiante tiene su propio usuario de Windows, puede usar --global como en la sección 6.2.

Al terminar cada sesión presencial

- Haga **git push** y compruebe en github.com que sus cambios llegaron.
- **Cierre su sesión de GitHub** en la computadora del laboratorio: menú Inicio → **Administrador de credenciales** → **Credenciales de Windows** → **git:https://github.com** → **Quitar**. Así nadie más puede subir cambios a su nombre.
- En la casa, antes de trabajar: **git pull**, para traer lo que hizo en el laboratorio.

Rutina entre la casa y el laboratorio

```
# Al empezar, en cualquier computadora
git pull
# ... trabajar y hacer commits ...
# Al terminar, en cualquier computadora
git push
```

9. Parte 2 · Node.js, TypeScript y Angular (antes de la sesión 26)

Node.js permite ejecutar JavaScript fuera del navegador e incluye **npm**, el instalador de paquetes con el que se instalan TypeScript y Angular. Puede instalarlo hoy si le sobra tiempo; es obligatorio a partir de la sesión 26.

9.1 Node.js

1. Entre a **nodejs.org** y descargue la versión **LTS** (soporte a largo plazo). En noviembre de 2026 es **Node.js 26** («Lithium»). No use la versión «Current».
2. Ejecute el instalador .msi con las opciones por defecto. Si ofrece «Tools for Native Modules», **no** es necesario marcarlo.
3. Cierre y vuelva a abrir VS Code. En la terminal verifique las versiones:

Terminal · Git Bash

```
node -v      # debe mostrar v26.x.x
npm -v       # muestra la versión de npm
```

9.2 TypeScript (sesión 26)

Terminal · Git Bash

```
npm install -g typescript
tsc -v       # debe mostrar Version 7.x
```

9.3 Angular CLI (sesión 32)

Terminal · Git Bash

```
npm install -g @angular/cli
ng version
```

> **Nota sobre versiones**
>
> Angular 22 (la versión actual) trabaja con Node.js 22, 24 o 26 y con **TypeScript 6.0**. No hay conflicto con el TypeScript 7 global: cada proyecto Angular instala **su propia** versión de TypeScript dentro de la carpeta node_modules y usa esa.

10. Atajos de VS Code que usará todos los días

Atajo (Windows)	macOS	Acción
`Ctrl+S`	`Cmd+S`	Guardar
`Ctrl+Shift+P`	`Cmd+Shift+P`	Paleta de comandos (buscar cualquier acción)
`Ctrl+P`	`Cmd+P`	Abrir un archivo por su nombre
`Ctrl+ñ` o `Ctrl+`` (según el teclado)	`Ctrl+``	Mostrar u ocultar la terminal
`Ctrl+/`	`Cmd+/`	Comentar o descomentar líneas

Atajo (Windows)	macOS	Acción
`Alt+↑ / Alt+↓`	`Option+↑ / Option+↓`	Mover la línea arriba o abajo
`Shift+Alt+↓`	`Shift+Option+↓`	Duplicar la línea
`Ctrl+D`	`Cmd+D`	Seleccionar la siguiente coincidencia (editar varias a la vez)
`Shift+Alt+F`	`Shift+Option+F`	Dar formato al documento
`! + Tab (en un .html vacío)`	`! + Tab`	Estructura HTML básica (Emmet, ya incluido)
`F12 (en el navegador)`	`Cmd+Option+I`	Abrir las herramientas del desarrollador

11. Solución de problemas

Problema	Solución
«code» no se reconoce como un comando	No se marcó «Agregar a PATH». Reinstale VS Code marcando esa casilla y reinicie la computadora.
No aparece «Open with Live Server» ni el botón Go Live	Abrió un archivo suelto. Use Archivo → Abrir carpeta…, o reinstale la extensión y recargue VS Code (Ctrl+Shift+P → «Recargar ventana»).
La página no cambia al editar	Guarde (Ctrl+S). Si sigue igual, recargue sin caché con Ctrl+F5.
En la consola: «Access to fetch… from origin 'null' has been blocked» o el Worker no carga	Abrió el archivo con doble clic (la dirección empieza con file://). Ábralo con Live Server (http://127.0.0.1:5500/…).
Tildes y ñ se ven como símbolos raros (Ã©)	Falta <meta charset="UTF-8"> en el HTML o el archivo no está en UTF-8 (véalo en la barra de estado; clic para «Volver a guardar con codificación» UTF-8).
git: «Author identity unknown»	Falta la sección 6.2: configure user.name y user.email.
git push pide usuario y contraseña	Cierre la ventana y vuelva a intentar; elija «Sign in with your browser». La contraseña de la cuenta ya no funciona en la terminal.
git push: «rejected… fetch first»	Hay cambios en GitHub que no tiene en su computadora (por ejemplo, editó el README en la web). Ejecute git pull y luego git push.
PowerShell: «npm.ps1 no se puede cargar porque la ejecución de scripts está deshabilitada»	Use Git Bash como terminal (sección 5). Si necesita PowerShell: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned.
El instalador pide permisos de administrador	En el laboratorio, avise a la persona facilitadora. En su casa, acepte el aviso de Windows.

12. Lista de verificación

✓	Comprobación	Cómo se verifica
	El navegador está actualizado y F12 abre las DevTools	Se ve la pestaña Console
	VS Code instalado y en español (opcional)	Menús en español

✓	Comprobación	Cómo se verifica
	El comando code funciona	code --version en la terminal muestra la versión
	Extensiones Live Server, Prettier y Error Lens instaladas	Aparecen en la vista Extensiones → Instaladas
	settings.json configurado	Al guardar un .html mal sangrado, se ordena solo
	Git instalado y configurado	git config --global --list muestra su nombre y correo
	Repositorio CSTI12004-Nombre-Apellido clonado en C:\INA	La barra de estado de VS Code muestra la rama main
	README principal y .gitignore subidos	La portada del repositorio en GitHub muestra el registro de evidencias
	Prueba del entorno funcionando con Live Server	La página dice «JavaScript está enlazado correctamente.»
	Cuadro comparativo en UA1/S01; prueba, captura y README de la S02 subidos	En GitHub, UA1/S02/README.md muestra la captura
	(S03) Repositorio clonado y prueba funcionando en la computadora del laboratorio	Live Server abre UA1/S02/prueba_entorno
	(S03) Sesión de GitHub cerrada en el laboratorio al terminar	No aparece git:https://github.com en el Administrador de credenciales
	(Parte 2) Node.js LTS instalado	node -v muestra v26.x.x

Comparta el URL de su repositorio con la persona facilitadora: es el que usará en el archivo **repositorio_url.txt** de cada proyecto del módulo.